UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Ingenieria y Arquitectura

Modelado y Control de Robots Moviles

Informe de Laboratorio – Capitulo 5 (Corke)

Navegacion de Robots Moviles: De lo Reactivo a ROS2 Nav2

Curso: Robotica
Codigo: MAN-4101136-1-2026-03

Integrantes:

Lina Robsana Jamioy Tisoy
Tomas Rocha Castano

Profesor:
Jaime Enrique Arango Castro

3 de noviembre de 2026

Índice

1. Objetivos **2**

2. Marco Teorico **2**

3. Entorno de Trabajo **3**

4. PARTE A: Navegacion Reactiva con Braitenberg **4**
4.1. Actividad A1: Implementacion 4

5. PARTE B: Navegacion con Bug2 **8**
5.1. Actividad B1: Simulacion 8

6. PARTE C: Planificacion con A* **11**
6.1. Actividad C1: Implementacion 11

7. PARTE D: Mapas de Ocupacion con Distance Transform **15**
7.1. Actividad D1: Implementacion 15

8. Discusion Comparativa **19**

9. Conclusiones **19**

10. PARTE E: Navegacion con Nav2 y Nodo Reactivo en ROS2 **20**
10.1. Objetivo de la parte 20
10.2. Actividad E1: Navegacion con Nav2 20
10.3. Actividad E2: Nodo reactivo 22

1. Objetivos

Al finalizar este laboratorio, el estudiante sera capaz de:

1. Implementar estrategias de navegacion reactiva basadas en sensores y campos escalares.
2. Analizar algoritmos de evasion de obstaculos (Bug2) y compararlos con el enfoque reactivo.
3. Implementar planificacion de trayectorias con A* y comparar su costo de busqueda con UCS.
4. Aplicar planificacion sobre mapas de ocupacion mediante la transformada de distancias.

2. Marco Teorico

La navegacion robotica guia un robot desde una posicion inicial hasta un objetivo. Existen dos enfoques principales:

- **Reactivo:** decide el movimiento a partir de sensores en el instante presente, sin un mapa global.
- **Basado en mapa:** planifica una ruta sobre una representacion del entorno, con el objetivo de optimizar algun criterio.

Este laboratorio cubre ambos enfoques. El vehiculo de Braitenberg es reactivo y actua sobre un campo escalar continuo. Bug2 es reactivo y se basa en el contacto con obstaculos. A* y la transformada de distancias planifican sobre una malla con conocimiento del mapa.

Campo escalar y ley de Braitenberg. El objetivo se representa con un campo

$$S(x, y) = \frac{1000}{(x - x_c)^2 + (y - y_c)^2 + 50} \tag{1}$$

que alcanza su maximo en la meta (x_c, y_c). Dos sensores separados lateralmente d miden S_L y S_R. La ley de control es

$$v = -K_v \, (S_L + S_R) \tag{2}$$

$$\omega = K_s \, (S_R - S_L) \tag{3}$$

$$\dot{x} = v \cos\theta, \qquad \dot{y} = v \sin\theta, \qquad \dot{\theta} = \omega \tag{4}$$

Algoritmo A*. Ordena la expansion de nodos con $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado desde el inicio y $h(n)$ una heuristica. Con la distancia euclidiana, en una malla de 4 vecinos, h es admisible y el camino obtenido es optimo.

Transformada de distancias. Para cada celda libre p se cumple

$$D(p) = \min\Big(D(p), \min_{q \in \mathcal{N}_4(p)} D(q) + 1\Big), \qquad D(\text{meta}) = 0 \tag{5}$$

Repetida hasta converger, $D(p)$ es la longitud del camino mas corto desde p hasta la meta.

3. Entorno de Trabajo

- **Sistema Operativo:** Ubuntu 24.04.5 LTS
- **Lenguaje:** Python 3
- **Librerias:** `numpy` 1.26.4, `matplotlib` 3.6.3, `scipy` 1.11.4
- **Herramientas de captura:** `gnome-screenshot` y `cp` desde la terminal

Los scripts se ejecutaron desde la carpeta `lab_nav_cap5`, con subcarpetas `codigo/`, `graficas/` y `capturas/`.

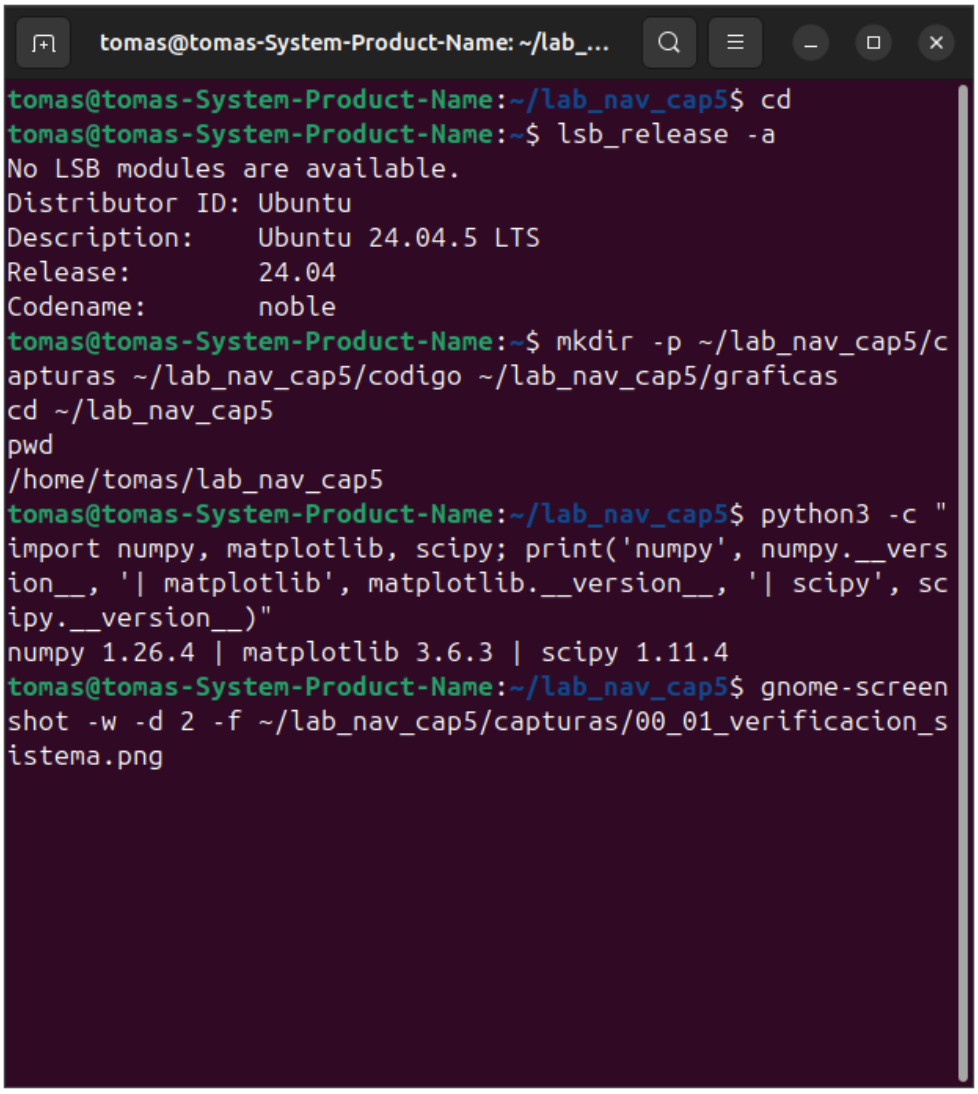


Figura 1: Verificacion del entorno: sistema operativo y versiones de las librerias en la terminal.

4. PARTE A: Navegacion Reactiva con Braitenberg

4.1. Actividad A1: Implementacion

Se implemento el vehiculo de Braitenberg sobre el campo escalar de la guia, con $K_s = 17$, $K_v = 14$, $\Delta t = 0{,}1$ s, separacion lateral de sensores $d = 2$ y estado inicial $(10, 10, 0)$. El objetivo se ubica en $(60, 90)$. Se simularon 222 iteraciones (como en la guia) y 600 iteraciones para analizar el comportamiento a largo plazo.

```python
# Braitenberg Vehicle - Parte A (Actividad A1)
import numpy as np
import matplotlib.pyplot as plt

# Campo escalar (objetivo)
def sensorfield(x, y):
    xc, yc = (60, 90)
    return 1000 / ((x - xc)**2 + (y - yc)**2 + 50)

# Parametros
dt = 0.1
Ks = 17
Kv = 14
d = 2  # separacion lateral de los sensores

# Estado inicial
x, y, theta = 10, 10, 0
traj_x, traj_y = [], []

for i in range(222):
    # Sensores (offset lateral)
    s_left = sensorfield(x - d*np.sin(theta), y + d*np.cos(theta))
    s_right = sensorfield(x + d*np.sin(theta), y - d*np.cos(theta))

    # Ley de control
    v = -Kv * (s_left + s_right)        # velocidad lineal
    omega = Ks * (s_right - s_left)     # velocidad angular

    # Cinematica
    x += v * np.cos(theta) * dt
    y += v * np.sin(theta) * dt
    theta += omega * dt

    traj_x.append(x)
    traj_y.append(y)

# Visualizacion
plt.figure()
plt.plot(traj_x, traj_y, label="trayectoria")
plt.scatter([10], [10], c='blue', label="inicio")
plt.scatter([60], [90], c='red', label="objetivo")
plt.title("Navegacion Braitenberg")
plt.axis('equal')
plt.grid(True)
plt.legend()
plt.savefig("graficas/A_trayectoria.png", dpi=150)
plt.show()
```

Listing 1: Script `a1_braitenberg.py`

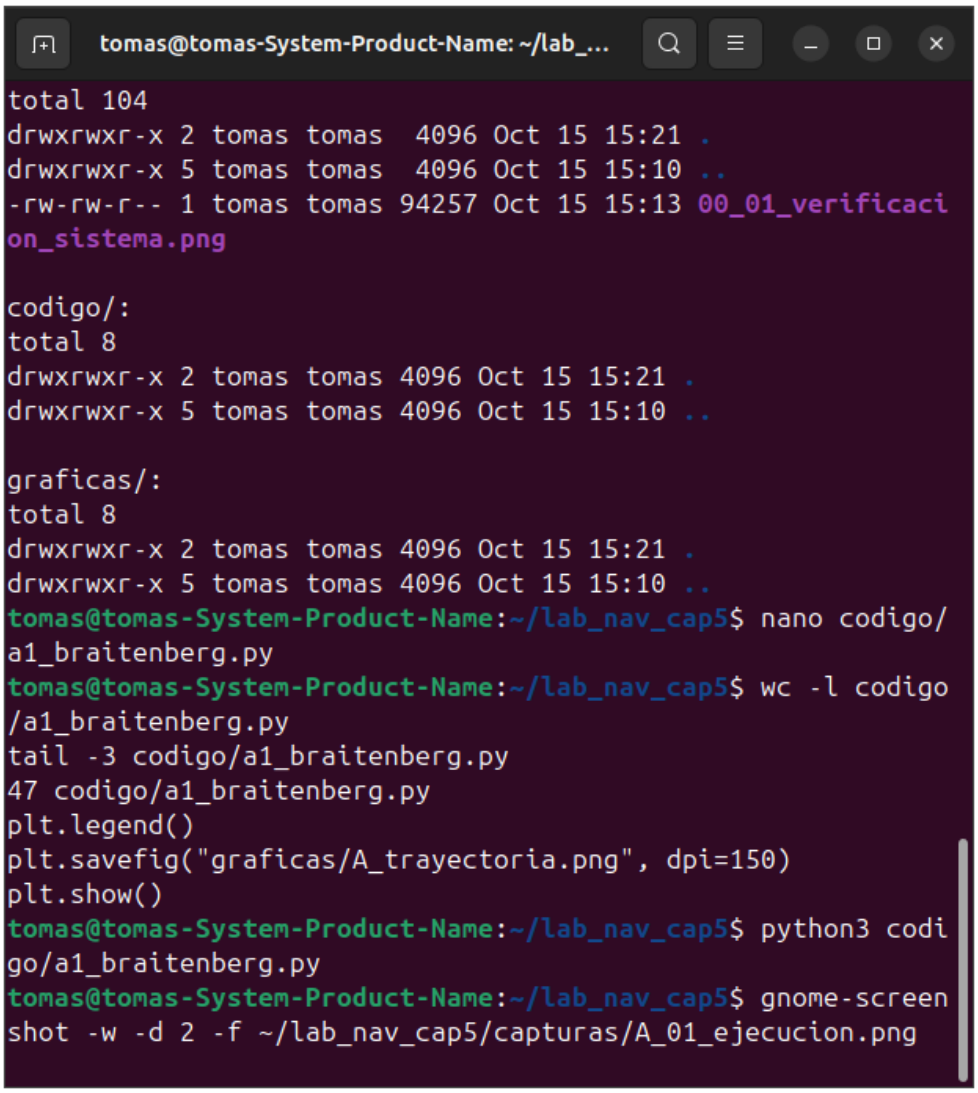


Figura 2: Ejecucion del script `a1_braitenberg.py` en la terminal. El script no imprime texto; la captura confirma la ejecucion.

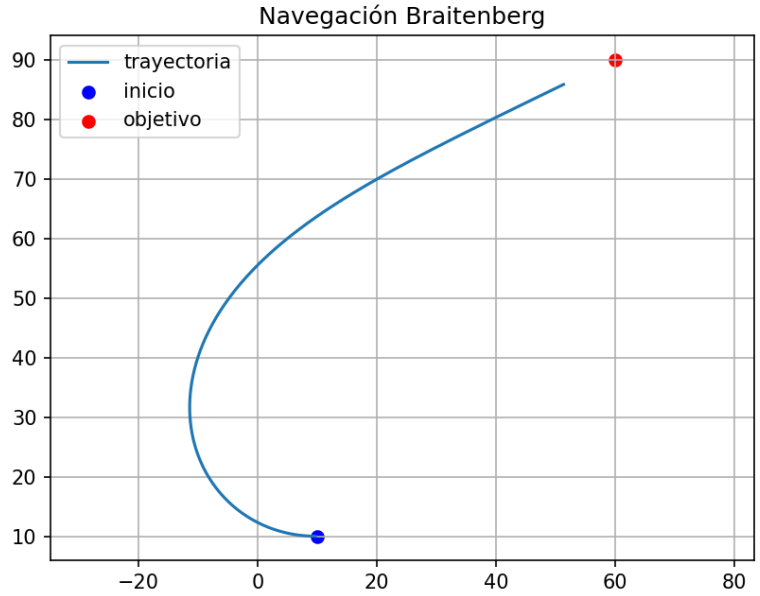


Figura 3: Trayectoria del vehiculo con 222 iteraciones desde $(10, 10)$. El robot termina cerca de $(51, 86)$, sin alcanzar el objetivo $(60, 90)$.

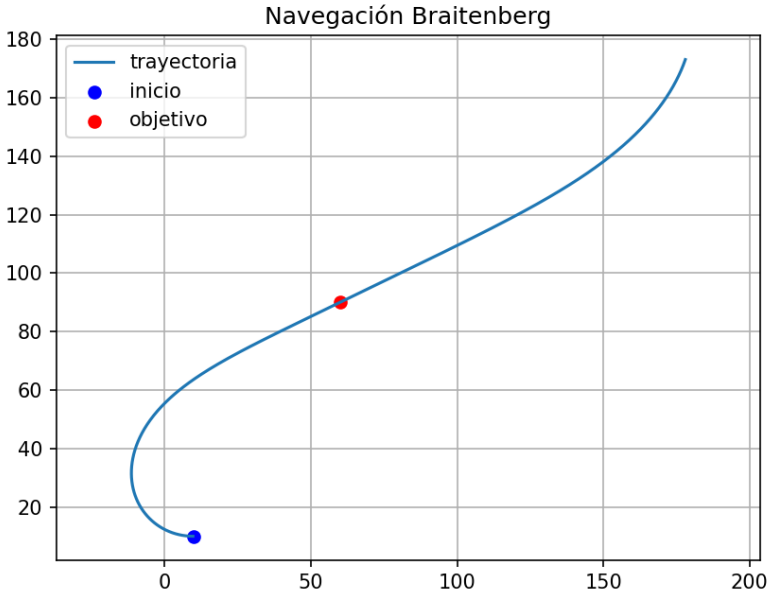


Figura 4: Trayectoria con 600 iteraciones. El robot cruza el objetivo $(60, 90)$ y continua hasta aproximadamente $(178, 173)$.

Preguntas y Analisis

1. **¿Como influye la diferencia entre sensores en la direccion del robot?**
   La velocidad angular es $\omega = K_s(S_R - S_L)$. Si el sensor derecho detecta mas intensidad, ω es positivo y el robot gira hacia la izquierda; si el izquierdo detecta mas, gira a la derecha. Si

ambos sensores leen lo mismo, $\omega = 0$ y el robot avanza en linea recta. La direccion depende solo de cual lado tiene mayor valor del campo.

2. **¿Que ocurre cuando el robot se aproxima al objetivo?**
El campo $1000/(d^2+50)$ crece al acercarse a la meta, y con el la magnitud de $v = -K_v(S_L + S_R)$. El robot acelera justo cuando deberia frenar, por eso no se detiene al llegar y sobrepasa el objetivo.

3. **¿Que limitaciones presenta este enfoque? (¿que pasa si aumentamos las iteraciones?)**

   - **No hay criterio de parada.** El control siempre produce movimiento, por lo que el robot nunca se queda en el objetivo.
   - **Sobrepaso.** Con 600 iteraciones el robot atraviesa el objetivo y se aleja hasta $(178, 173)$.
   - **Solo mide intensidad.** El robot no conoce la direccion ni la distancia al objetivo, solo el valor del campo en su posicion.
   - **Sin memoria ni mapa.** No recuerda su recorrido y no puede planificar ante obstaculos.
   - **Dependencia de parametros.** K_s, K_v y Δt determinan si la trayectoria es suave o inestable.

5. PARTE B: Navegacion con Bug2

5.1. Actividad B1: Simulacion

Se creo un mapa tipo grid de 20×20 con un obstaculo en las filas $y = 8$–11 y columnas $x = 5$–14. El robot parte de $(2, 2)$ y busca la meta $(18, 18)$. Para la evasion, el script usa la estrategia de la guia: avanzar hacia la meta por la direccion dominante y, si la celda siguiente esta ocupada, probar derecha, arriba, izquierda y abajo, en ese orden.

Nota sobre la implementacion. El codigo es una **version simplificada** de Bug2. Un Bug2 completo usa la linea recta inicio-meta (la *m-line*) y deja de seguir el borde cuando vuelve a cruzarla mas cerca de la meta. Esta implementacion no tiene m-line ni criterio de regreso a la linea; el comportamiento depende del orden fijo de direcciones y del desempate `abs(dx) >abs(dy)`.

```
# Bug2 simplificado en grid - Parte B (Actividad B1)
import numpy as np
import matplotlib.pyplot as plt

grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1          # obstaculo (filas y=8..11, columnas x=5..14)

start = np.array([2, 2])
goal = np.array([18, 18])
pos = start.copy()
path = [pos.copy()]

def is_free(p):
    return grid[p[1], p[0]] == 0

for _ in range(500):
    direccion = goal - pos
    if abs(direccion[0]) > abs(direccion[1]):
        step = np.array([np.sign(direccion[0]), 0])
    else:
        step = np.array([0, np.sign(direccion[1])])

    next_pos = pos + step
    if is_free(next_pos):
        pos = next_pos
    else:
        for move in [np.array([1, 0]), np.array([0, 1]), np.array([-1, 0]), np.
    array([0, -1])]:
            if is_free(pos + move):
                pos = pos + move
                break

    path.append(pos.copy())
    if np.linalg.norm(pos - goal) < 1:
        break

path = np.array(path)
print("Pasos realizados:", len(path) - 1)
print("Posicion final:", path[-1], "| objetivo alcanzado:", np.array_equal(path
    [-1], goal))

plt.figure()
plt.imshow(grid, origin='lower', cmap='gray_r')
plt.plot(path[:, 0], path[:, 1], 'g', label='trayectoria')
plt.scatter(start[0], start[1], c='blue', label='inicio')
plt.scatter(goal[0], goal[1], c='red', label='objetivo')
plt.title("Bug2 simplificado")
```

```
plt.legend()
plt.savefig("graficas/B_trayectoria.png", dpi=150)
plt.show()
```

Listing 2: Script `b1_bug2.py` (escenario 1)

Para evidenciar la evasion se creo una variante (escenario 2) cambiando la meta a $(18, 2)$ y el obstaculo a las filas $y = 1$–3 y columnas $x = 8$–11, justo sobre la linea recta.

```
tomas@tomas-System-Product-Name:~/lab_nav_cap5$ cd ~/lab_nav
_cap5
tomas@tomas-System-Product-Name:~/lab_nav_cap5$ nano codigo/
b1_bug2.py
tomas@tomas-System-Product-Name:~/lab_nav_cap5$ wc -l codigo
/b1_bug2.py
48 codigo/b1_bug2.py
tomas@tomas-System-Product-Name:~/lab_nav_cap5$ tail -3 codi
go/b1_bug2.py
plt.legend()
plt.savefig("graficas/B_trayectoria.png", dpi=150)
plt.show()
tomas@tomas-System-Product-Name:~/lab_nav_cap5$ python3 codi
go/b1_bug2.py
Pasos realizados: 32
Posición final: [18 18] | objetivo alcanzado: True
tomas@tomas-System-Product-Name:~/lab_nav_cap5$ gnome-screen
shot -w -d 2 -f ~/lab_nav_cap5/capturas/B_01_ejecucion.png
tomas@tomas-System-Product-Name:~/lab_nav_cap5$ cp graficas/
B_trayectoria.png capturas/B_02_trayectoria.png
```

Figura 5: Ejecucion de `b1_bug2.py` en la terminal. Salida: 32 pasos y objetivo alcanzado.

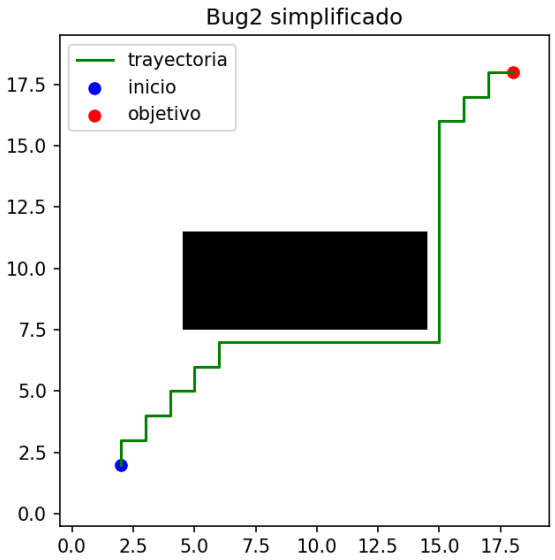


Figura 6: Escenario 1: trayectoria de Bug2 simplificado con 32 pasos. El robot pasa por debajo del obstaculo sin tocarlo.

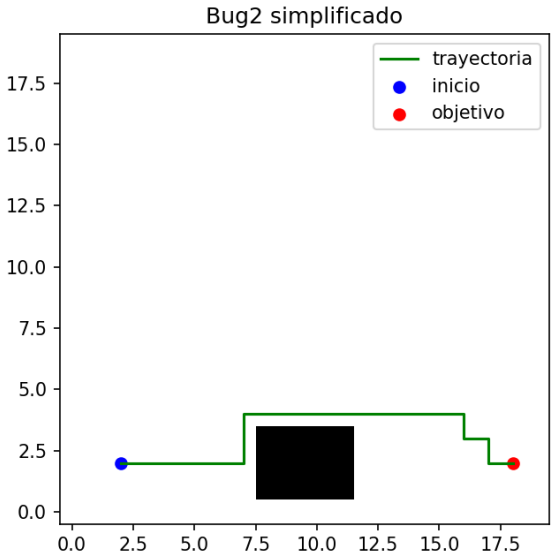


Figura 7: Escenario 2: el robot detecta el obstaculo en $x = 7$, lo rodea por arriba y alcanza la meta en 20 pasos.

Resultados

- **Escenario 1** (meta $(18, 18)$): 32 pasos, objetivo alcanzado. El obstaculo nunca se toca, por lo que el bloque de evasion no se ejecuta.
- **Escenario 2** (meta $(18, 2)$): 20 pasos, objetivo alcanzado. Aqui si se activa la evasion y el robot rodea el obstaculo por arriba.

Preguntas y Analisis

1. **¿El camino obtenido es optimo? Justifique.**
   En el escenario 2 si: 16 pasos en linea recta mas 2 de subida y 2 de bajada dan 20, que es el minimo para esa malla de 4 vecinos. En el escenario 1 la longitud (32) tambien es la minima, pero es la distancia Manhattan entre inicio y meta: el camino es monotono y el obstaculo no lo obliga a alargarse. El algoritmo no garantiza optimalidad en general, porque depende del orden fijo de direcciones.

2. **¿Que sucede en entornos complejos?**
   Sin m-line, el robot puede quedar atrapado en minimos locales u oscilar entre dos celdas, o bien recorrer rodeos muy largos. El limite de 500 iteraciones es lo unico que detiene la simulacion en esos casos.

3. **Compare con el enfoque reactivo de la Parte A.**
   Ambos son reactivos y no usan un mapa global. Braitenberg actua sobre un campo continuo, nunca se detiene y sobrepasa el objetivo. Bug2 es discreto, reacciona al contacto con el obstaculo y tiene un criterio de parada claro al llegar a la meta.

6. PARTE C: Planificacion con A*

6.1. Actividad C1: Implementacion

Se uso el mismo mapa que en la Parte B (inicio $(2, 2)$, meta $(18, 18)$, obstaculo en $y = 8$–11). La heuristica es la distancia euclidiana. Para medir el efecto de la heuristica, el script ejecuta A* y UCS (A* con $h = 0$) sobre el mismo mapa y compara el numero de nodos extraidos de la cola de prioridad.

```python
# A* desde cero - Parte C (Actividad C1)
import heapq
import numpy as np
import matplotlib.pyplot as plt

grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1          # obstaculo (filas y=8..11, columnas x=5..14)

start = (2, 2)
goal = (18, 18)

def heuristica(a, b):
    return np.linalg.norm(np.array(a) - np.array(b))

def astar(grid, start, goal, usar_heuristica=True):
    open_list = []
    heapq.heappush(open_list, (0, start))
    came_from = {}
    cost = {start: 0}
    expandidos = 0

    while open_list:
        _, current = heapq.heappop(open_list)
        expandidos += 1
        if current == goal:
            break

        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            neighbor = (current[0] + dx, current[1] + dy)
            if (0 <= neighbor[0] < grid.shape[1] and
                    0 <= neighbor[1] < grid.shape[0] and
                    grid[neighbor[1], neighbor[0]] == 0):
                new_cost = cost[current] + 1
                if neighbor not in cost or new_cost < cost[neighbor]:
                    cost[neighbor] = new_cost
                    h = heuristica(neighbor, goal) if usar_heuristica else 0
                    heapq.heappush(open_list, (new_cost + h, neighbor))
                    came_from[neighbor] = current

    # reconstruir camino
    path = [goal]
    node = goal
    while node != start:
        node = came_from[node]
        path.append(node)
    path.reverse()
    return path, expandidos

path, exp_astar = astar(grid, start, goal, usar_heuristica=True)
_, exp_ucs = astar(grid, start, goal, usar_heuristica=False)
path = np.array(path)

print("Longitud del camino (pasos):", len(path) - 1)
print("Nodos expandidos A*:", exp_astar)
```

```
print("Nodos expandidos UCS (sin heuristica):", exp_ucs)

plt.figure()
plt.imshow(grid, origin='lower', cmap='gray_r')
plt.plot(path[:, 0], path[:, 1], 'r', linewidth=2, label='A*')
plt.scatter(start[0], start[1], c='blue', label='inicio')
plt.scatter(goal[0], goal[1], c='red', label='objetivo')
plt.title("A* Path")
plt.legend()
plt.savefig("graficas/C_ruta.png", dpi=150)
plt.show()
```

Listing 3: Script `c1_astar.py`

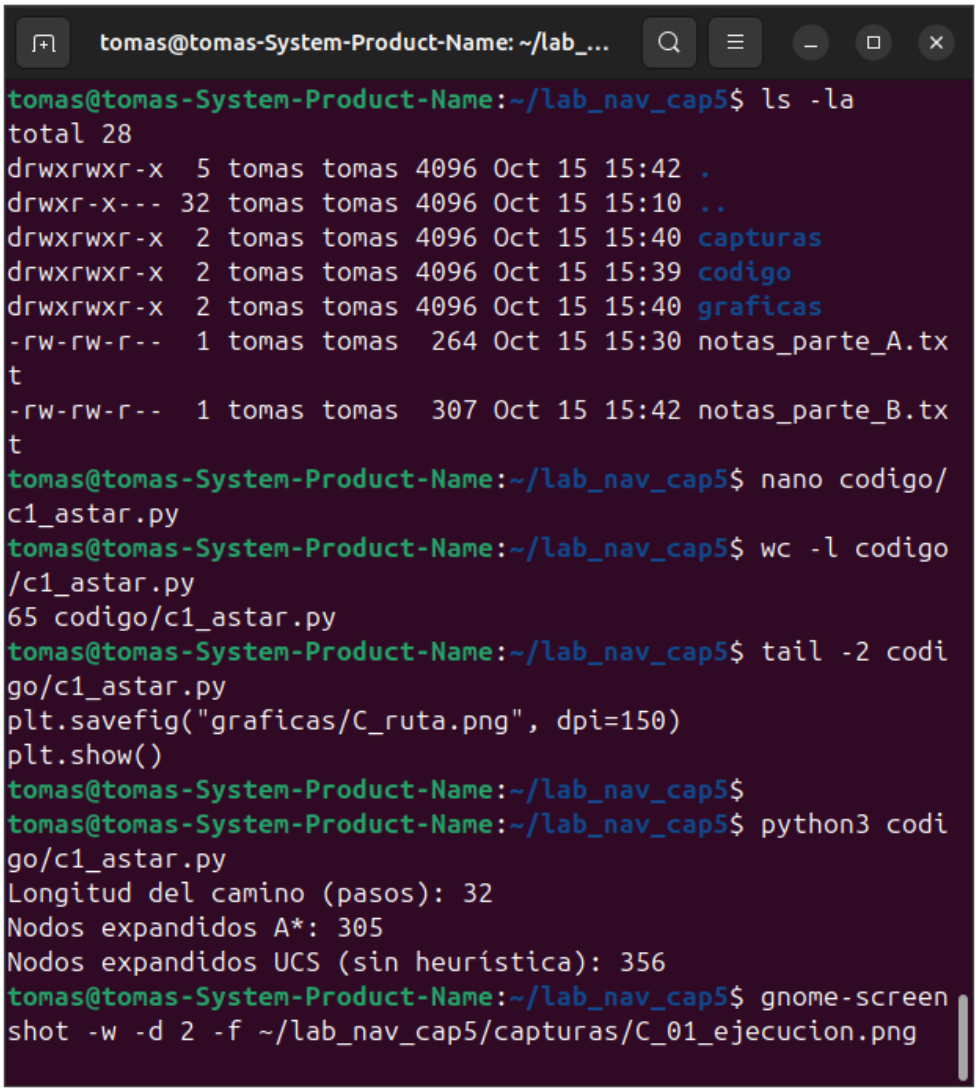


Figura 8: Ejecucion de `c1_astar.py`. Salida: longitud 32, 305 nodos expandidos por A* y 356 por UCS.

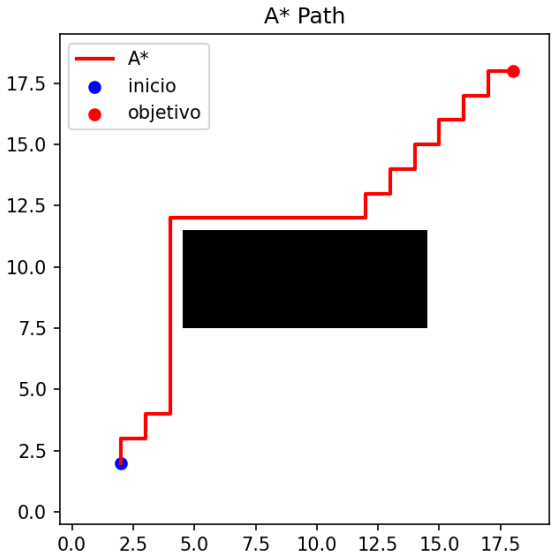


Figura 9: Ruta obtenida con A* desde $(2, 2)$ hasta $(18, 18)$. La ruta tiene 32 pasos y rodea el obstaculo por arriba.

Resultados

- **Longitud del camino:** 32 pasos, igual a la distancia Manhattan minima.
- **Nodos expandidos:** A* 305, UCS 356. El conteo incluye entradas duplicadas en la cola, porque el codigo no descarta entradas obsoletas; es un indicador razonable, no el numero exacto de nodos unicos.
- **Ruta:** sube por $x = 4$ hasta $y = 12$, avanza por $y = 12$ hasta $x = 12$ y sube en diagonal escalonada hasta la meta. Pasa por encima del obstaculo, mientras que Bug2 (escenario 1) pasa por debajo. Ambas son monotonas y tienen la misma longitud.

Preguntas y Analisis

1. **¿Que diferencia hay entre BFS, UCS y A*?**
   BFS y UCS son busquedas no informadas: expanden nodos segun el costo acumulado. En una malla de costo unitario, UCS equivale en la practica a BFS. A* ordena la cola por $g(n) + h(n)$ y prioriza los nodos que parecen acercarse a la meta. Con los datos del script, UCS expandio 356 nodos y A* 305. La reduccion es moderada porque el mapa es pequeno (20×20).

2. **¿Cual es el papel de la heuristica?**
   Guia la busqueda hacia la meta. La distancia euclidiana es admisible: nunca sobrestima el costo real en una malla de 4 vecinos, por lo que A* garantiza un camino optimo. Es una cota debil, porque el costo real en 4 vecinos es siempre al menos la distancia Manhattan, que es mayor que la euclidiana. Una heuristica Manhattan expandiria menos nodos sin perder optimalidad.

3. **¿Que ocurre si la heuristica no es adecuada?**

   - Si **sobrestima** el costo (por ejemplo, el doble de la euclidiana), A* puede expandir menos nodos, pero el camino ya no esta garantizado como optimo.

- Si **subestima** mucho, se comporta como UCS: el camino sigue siendo optimo, pero se expanden mas nodos.
- Si es **nula** ($h = 0$), el algoritmo es exactamente UCS.

7. PARTE D: Mapas de Ocupacion con Distance Transform

7.1. Actividad D1: Implementacion

Se uso el mismo mapa de ocupacion de las partes B y C. Esta vez el objetivo es la celda $(2, 2)$ y el punto inicial $(18, 18)$. La propagacion parte del objetivo y se repite 100 veces sobre toda la malla. Despues, el robot desciende siempre al vecino de menor valor hasta llegar a la meta.

```python
# Distance Transform - Parte D (Actividad D1)
import numpy as np
import matplotlib.pyplot as plt

# ========================= MAPA =========================
grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1          # obstaculo (filas y=8..11, columnas x=5..14)

goal = (2, 2)                  # celda objetivo (x, y)
start = (18, 18)               # punto inicial (x, y)

vecinos = [(-1, 0), (1, 0), (0, -1), (0, 1)]

# ======================== DISTANCE TRANSFORM ========================
dist = np.full(grid.shape, np.inf)
dist[goal[1], goal[0]] = 0

# propagacion iterativa desde el objetivo
for _ in range(100):
    for y in range(grid.shape[0]):
        for x in range(grid.shape[1]):
            if grid[y, x] == 0 and (x, y) != goal:
                for dx, dy in vecinos:
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < grid.shape[1] and 0 <= ny < grid.shape[0]:
                        dist[y, x] = min(dist[y, x], dist[ny, nx] + 1)

# ========================= SEGUIMIENTO DEL CAMINO =========================
pos = start
path = [pos]
for _ in range(200):
    if pos == goal:
        break
    x, y = pos
    candidatos = []
    for dx, dy in vecinos:
        nx, ny = x + dx, y + dy
        if 0 <= nx < grid.shape[1] and 0 <= ny < grid.shape[0] and grid[ny, nx]
    == 0:
            candidatos.append(((nx, ny), dist[ny, nx]))
    pos = min(candidatos, key=lambda t: t[1])[0]
    path.append(pos)

path = np.array(path)
print("Longitud del camino (pasos):", len(path) - 1)
print("Distancia en el mapa desde el inicio:", dist[start[1], start[0]])

# ========================= VISUALIZACION =========================
cmap = plt.cm.hot.copy()
cmap.set_bad('gray')                        # obstaculos en gris
dist_vis = np.where(grid == 1, np.nan, dist)

plt.figure(figsize=(6, 6))
plt.imshow(dist_vis, origin='lower', cmap=cmap)
plt.colorbar(label="Distancia al objetivo")
```

```
plt.plot(path[:, 0], path[:, 1], 'cyan', linewidth=2, label="trayectoria")
plt.scatter(start[0], start[1], c='blue', s=80, label="inicio")
plt.scatter(goal[0], goal[1], c='green', s=80, label="objetivo")
plt.title("Distance Transform - Mapa de distancias")
plt.legend()
plt.savefig("graficas/D_distancias.png", dpi=150)
plt.show()
```

Listing 4: Script `d1_distance_transform.py`

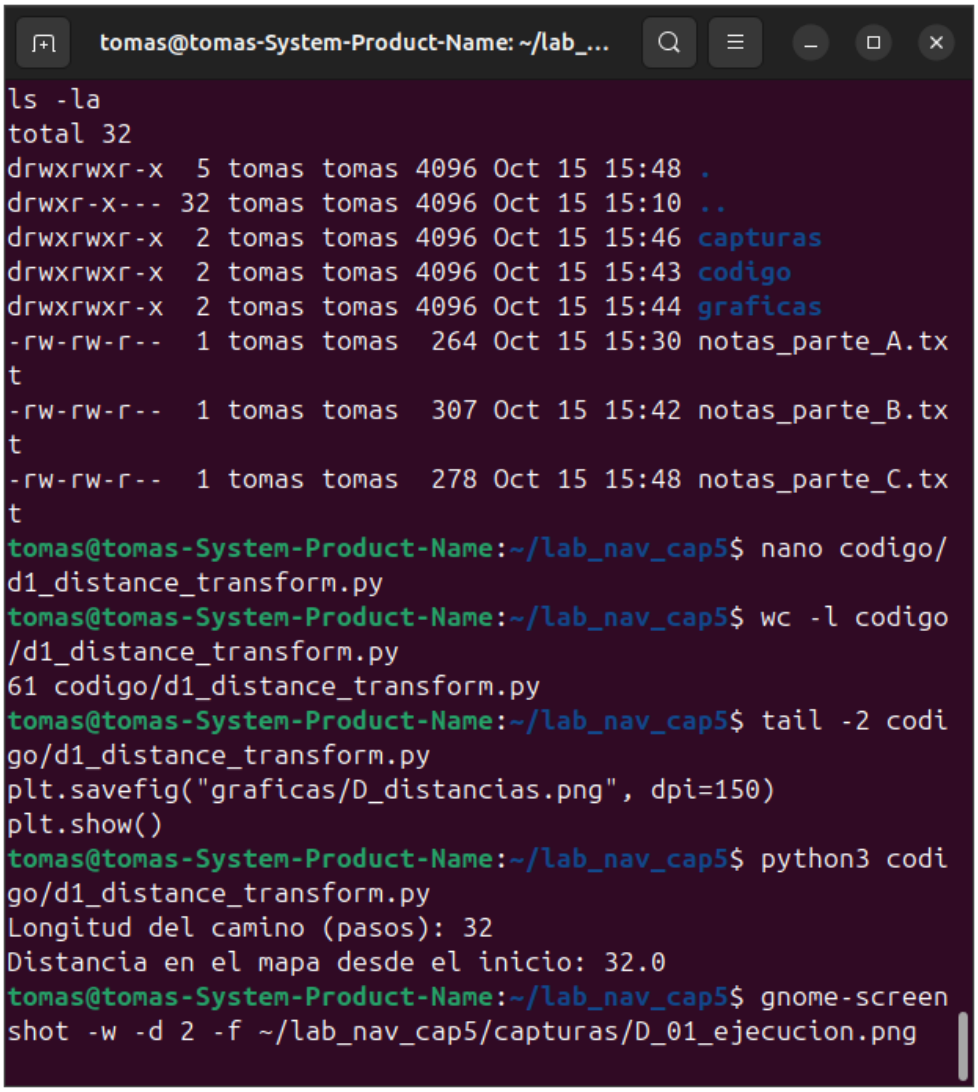


Figura 10: Ejecucion de `d1_distance_transform.py`. Salida: longitud 32 y distancia 32.0 en el inicio.

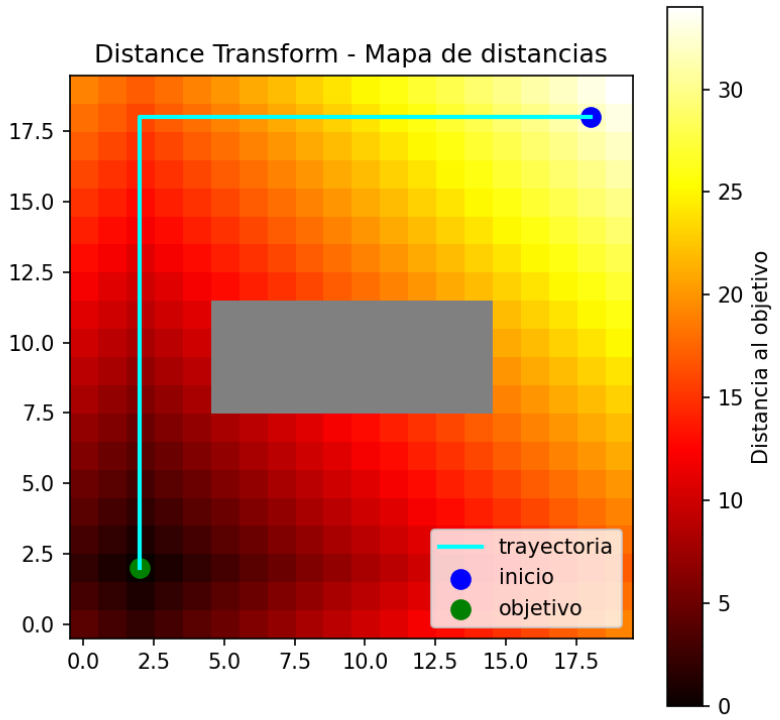


Figura 11: Mapa de distancias sobre la malla 20×20 con objetivo en $(2, 2)$. Las celdas grises son obstaculos. La trayectoria cyan parte de $(18, 18)$ y tiene 32 pasos.

Resultados

El valor del mapa en el inicio (32.0) coincide con la longitud del camino (32 pasos), lo que confirma que la propagacion convergio. La trayectoria no toca el obstaculo. El trazo horizontal-vertical se explica por el desempate: `min` elige el primer candidato en el orden de la lista de vecinos, que empieza por la izquierda.

Diferencia con la figura de la guia. La figura de la guia muestra un campo con flechas (*flow field*) y otros puntos de inicio y meta. El codigo que se ejecuto no dibuja flechas y usa los puntos indicados arriba.

Preguntas y Analisis

1. **¿Por que este metodo garantiza optimalidad?**
   La propagacion calcula, para cada celda libre, el minimo entre su valor actual y el de sus vecinos mas uno. Repetida hasta converger, cada valor es la longitud del camino mas corto hasta la meta. El seguimiento baja siempre al vecino de menor valor, y cada paso reduce la distancia en exactamente 1, por lo que el camino resultante es minimo.

2. **¿Cual es su costo computacional?**
   Cada pasada recorre las N celdas de la malla, con costo $O(N)$. El codigo hace 100 pasadas fijas, es decir $O(100N)$, sin comprobar convergencia. En el peor caso, por ejemplo en un

laberinto, el numero de pasadas necesarias puede crecer mucho. Un BFS desde la meta cuesta $O(N)$ en total, porque cada celda se visita una vez.

3. **Compare con el algoritmo Bug2.**

Aspecto	**Bug2 (Parte B)**	**Distance Transform (Parte D)**
Informacion	Reactiva: solo detecta contacto	Requiere el mapa completo
Optimalidad	No garantizada	Garantizada
Costo	Muy bajo en memoria y calculo	Calcula todo el mapa
Reutilizacion	Un nuevo objetivo exige volver a correr	Un mapa sirve para todos los inicios
Criterio de parada	Llega a la meta	Converge y luego sigue el gradiente

Bug2 sirve cuando no hay mapa; la transformada de distancias sirve cuando el mapa es conocido y se necesitan rutas optimas desde muchos puntos.

8. Discusion Comparativa

Metodo	**Informacion**	**Resultado (escenario de la Parte B/C/D)**	**Optimalidad**
Braitenberg (A)	Campo local, sin mapa	No se detiene; sobrepasa la meta	No aplica
Bug2 (B)	Contacto con obstaculos	32 pasos (esc. 1); 20 pasos (esc. 2)	No garantizada
A* (C)	Mapa completo	32 pasos; 305 nodos expandidos	Garantizada (heuristica admisible)
Distance transform (D)	Mapa completo	32 pasos; mapa para todos los inicios	Garantizada

1. **¿Que enfoque es adecuado para cada situacion?**
   Los metodos reactivos (A y B) son adecuados cuando el robot no tiene mapa o el entorno cambia. Los metodos basados en mapa (C y D) son preferibles cuando el entorno es conocido y se quiere una ruta optima. Entre ellos, A* conviene para un par inicio-meta y la transformada de distancias para muchos inicios hacia una misma meta.

2. **¿Que limitaciones comunes tienen las simulaciones?**
   Todas usan mallas pequenas (20×20) y un solo obstaculo. Los resultados no permiten generalizar el rendimiento a entornos complejos. Ademas, las simulaciones no incluyen ruido de sensores ni dinamica del robot.

9. Conclusiones

1. El vehiculo de Braitenberg navega hacia el objetivo mediante un campo escalar y dos sensores, pero no tiene criterio de parada: con mas iteraciones sobrepasa la meta.

2. La version simplificada de Bug2 alcanza la meta en ambos escenarios. Solo en el escenario 2 se activa la evasion de obstaculos, con 20 pasos, optimo para esa malla. La implementacion no es un Bug2 completo porque carece de m-line.

3. A* obtiene un camino optimo de 32 pasos y expande menos nodos que UCS (305 frente a 356) gracias a la heuristica.

4. La transformada de distancias produce un mapa de costos coherente: el valor en el inicio coincide con la longitud del camino. Su costo crece con el numero de pasadas necesarias.

Referencias

1. P. Corke, *Robotics, Vision and Control*, Springer. Capitulo 5: Navegacion.

2. Guia de laboratorio “Simulacion Capitulo 5 – Navegacion de Robots Moviles: De lo Reactivo a ROS2 Nav2”.

10. PARTE E: Navegacion con Nav2 y Nodo Reactivo en ROS2

10.1. Objetivo de la parte

En esta parte el robot ya no trabaja sobre una cuadricula de juguete, sino en un simulador completo. Usamos ROS2 Jazzy con Gazebo y el robot TurtleBot3. Hicimos dos cosas: primero, dejar que el sistema Nav2 llevara al robot a una meta usando un mapa; y segundo, programar un nodo sencillo que reacciona al láser sin mapa.

10.2. Actividad E1: Navegacion con Nav2

Preparacion. Instalamos Nav2, TurtleBot3 y el puente entre ROS2 y Gazebo. Verificamos que ROS2 Jazzy estaba activo y que el modelo del robot estaba configurado. La Figura 12 muestra esta verificacion.

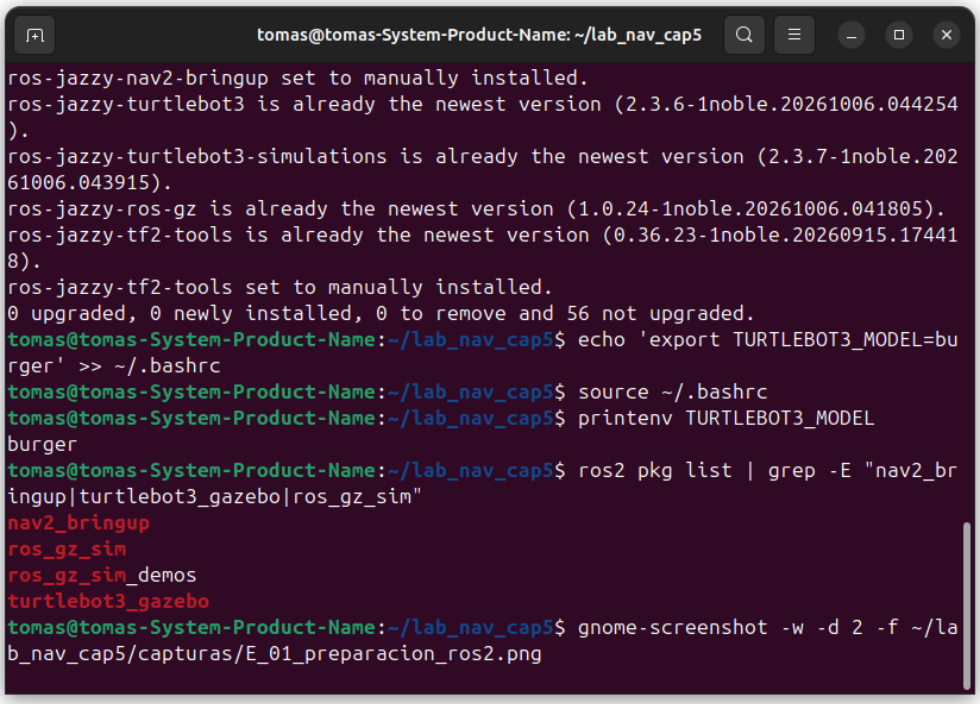


Figura 12: Verificacion de ROS2 Jazzy, instalacion de Nav2 y TurtleBot3.

Puesta en marcha. Lanzamos la simulacion con Nav2 y abrimos RViz. Antes de enviar metas, fijamos la pose inicial del robot con la herramienta *2D Pose Estimate*. Esto le indica al sistema de localizacion (AMCL) donde esta el robot en el mapa. La Figura 13 muestra el robot en Gazebo y la Figura 14 muestra RViz con la localizacion ya estimada.

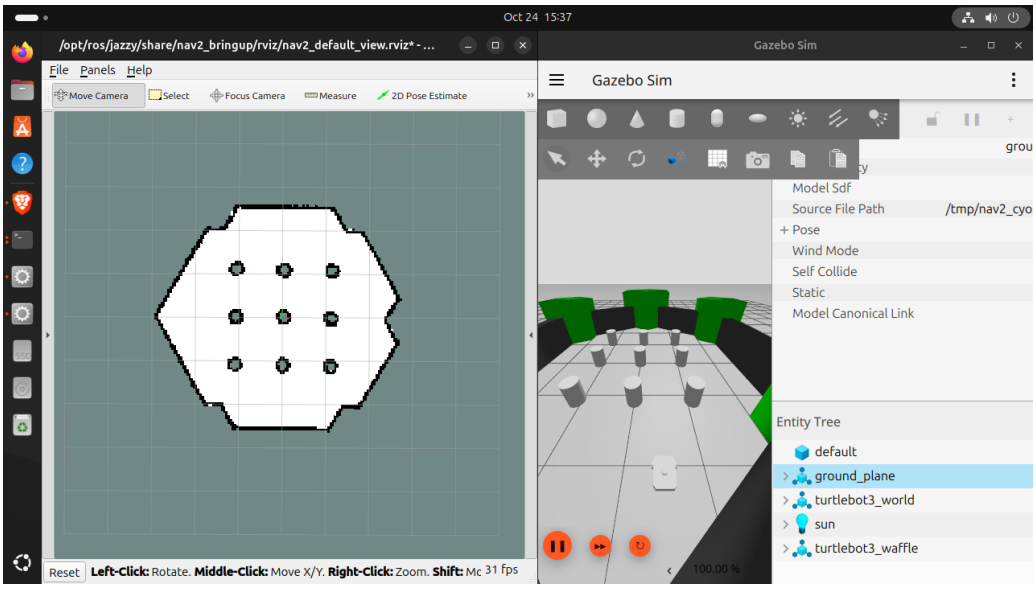


Figura 13: Robot TurtleBot3 en el mundo de Gazebo.

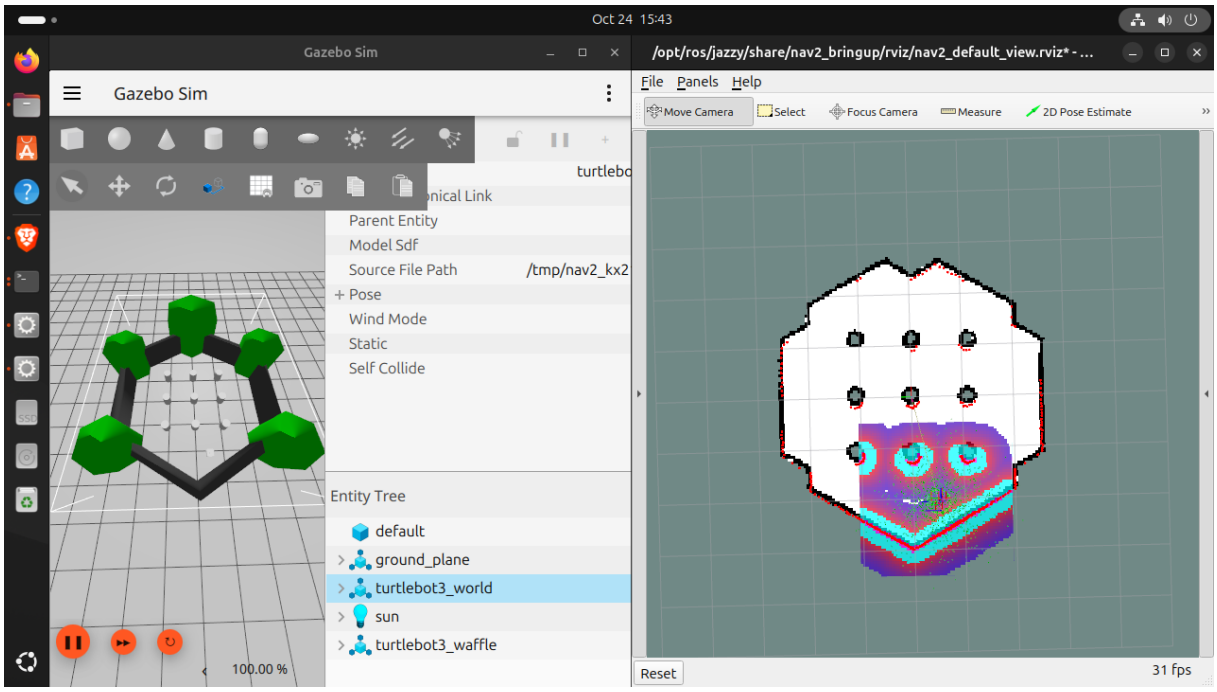


Figura 14: Gazebo y RViz al mismo tiempo, con el mapa cargado y el láser activo tras fijar la pose inicial.

Verificacion del sistema. Revisamos que los nodos de Nav2 estuvieran activos. El planificador, el controlador y AMCL respondieron `active [3]`. Comprobamos tambien que la transformacion entre el mapa y el robot existia, y que la pose de AMCL coincidia con la posicion del robot en Gazebo.

Meta y ruta global. Enviamos una meta con *Nav2 Goal*. El planificador calculo una ruta global, que aparece en RViz como una linea de color, y la verificamos con el topico `/plan`. La Figura 15 muestra la ruta trazada hacia la meta.

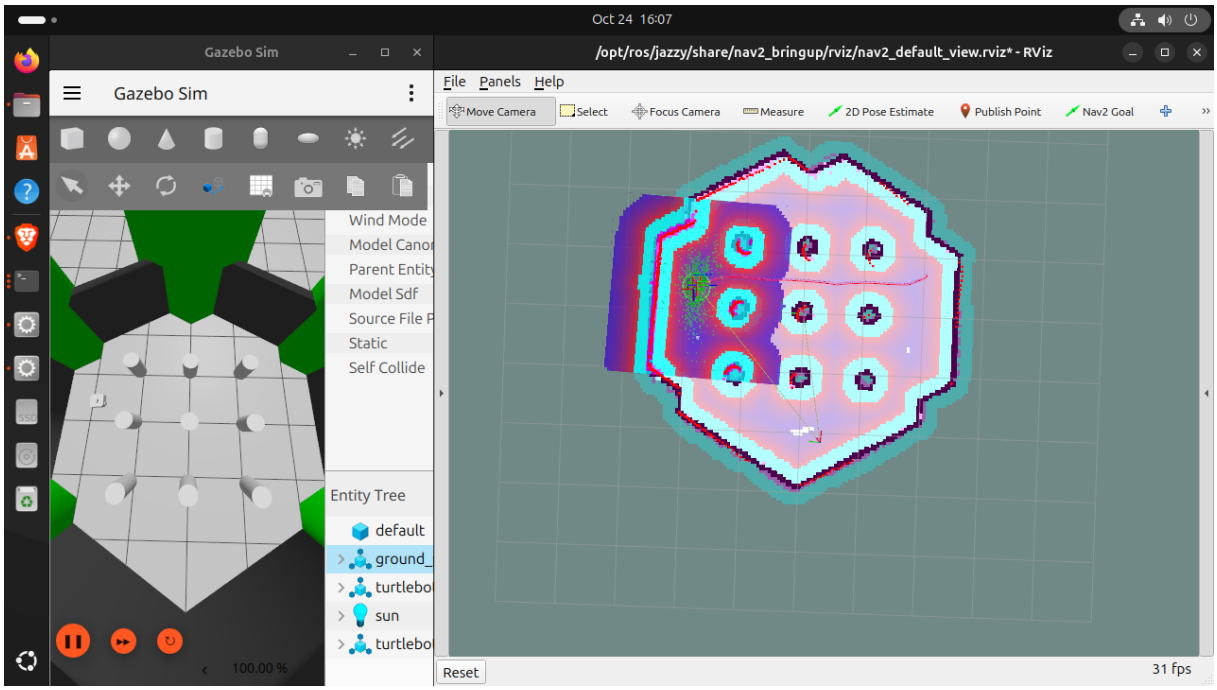


Figura 15: RViz con la ruta global trazada hacia la meta enviada.

El video del movimiento completo esta en el siguiente enlace:

[https://youtu.be/go3Mom6UoRg]

10.3. Actividad E2: Nodo reactivo

El nodo reactivo es una version sencilla de la guia que lee el láser y decide la velocidad sin usar mapa. Su logica es:

- Mira las lecturas del láser en un sector de $\pm 30°$ al frente del robot.
- Si el obstaculo mas cercano esta a menos de 0,5 m, frena y gira a la izquierda.
- Si no hay obstaculo cerca, avanza a 0,2 m/s.

Cambios respecto a la guia. La guia publica en `/cmd_vel` con un tipo de mensaje que en nuestra simulacion se usa como `TwistStamped`, y el sector frontal que proponia tomaba el centro del arreglo, que corresponde a la parte trasera del robot en este sensor. Por eso ajustamos el sector a los extremos del arreglo, donde estan los angulos cercanos a cero.

```
# Nodo reactivo - Parte E (Actividad E2)
# /cmd_vel usa TwistStamped porque el puente ros_gz de esta simulaci n lo
    espera as .
# angle_min = 0, angle_max = 2*pi, 1 grado por muestra: el frente est  al
    inicio y al final del arreglo.
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import TwistStamped
import numpy as np

FRONT_HALF_WIDTH = 30    # muestras a cada lado de 0 rad (~30 grados)
OBSTACLE_DIST = 0.5      # metros

```

```python
class ReactiveNavigator(Node):
    def __init__(self):
        super().__init__('reactive_navigator')
        self.sub = self.create_subscription(LaserScan, '/scan', self.
    scan_callback, 10)
        self.pub = self.create_publisher(TwistStamped, '/cmd_vel', 10)

    def make_cmd(self, linear, angular):
        cmd = TwistStamped()
        cmd.header.stamp = self.get_clock().now().to_msg()
        cmd.header.frame_id = 'base_link'
        cmd.twist.linear.x = linear
        cmd.twist.angular.z = angular
        return cmd

    def scan_callback(self, msg):
        ranges = np.array(msg.ranges, dtype=float)
        k = FRONT_HALF_WIDTH
        front_vals = np.concatenate((ranges[-k:], ranges[:k]))
        front_vals = front_vals[np.isfinite(front_vals)]
        front = np.min(front_vals) if len(front_vals) > 0 else np.inf

        if front < OBSTACLE_DIST:
            self.pub.publish(self.make_cmd(0.0, 0.8))   # frenar y girar a la
    izquierda
        else:
            self.pub.publish(self.make_cmd(0.2, 0.0))   # avanzar

def main():
    rclpy.init()
    node = ReactiveNavigator()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.pub.publish(node.make_cmd(0.0, 0.0))   # detener el robot al salir
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Listing 5: Script `e2_reactive_navigator.py`

Resultado. Ejecutamos el nodo con Nav2 apagado, para que solo un controlador publicara en `/cmd_vel`. La Figura 16 muestra la simulacion durante la ejecucion.

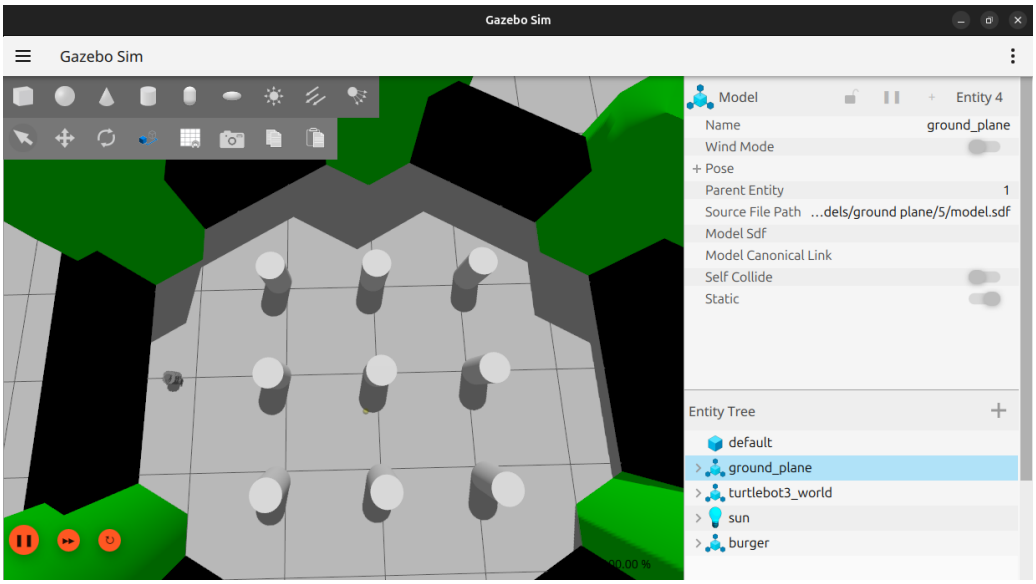


Figura 16: Gazebo durante la ejecucion del nodo reactivo.

El video de la prueba esta en el siguiente enlace:

[https://youtu.be/YQFfcxXbcW4]

Preguntas y Analisis

1. **¿Que diferencia hay entre navegacion reactiva y Nav2?**
   La navegacion reactiva decide con lo que el sensor ve en ese instante. Nav2 usa un mapa, sabe donde esta el robot (localizacion) y planifica una ruta completa hacia la meta. El nodo reactivo es simple y rapido, pero no recuerda ni planea; Nav2 es mas completo, pero necesita mas informacion.

2. **¿Que funcion cumple el planificador dentro de Nav2?**
   El planificador calcula la ruta global desde la posicion actual hasta la meta, sobre el mapa. Esa ruta es la que el controlador sigue, ajustando la velocidad para evitar obstaculos cercanos.

3. **¿Que ventajas ofrece ROS2 en robotica movil?**
   ROS2 separa el sistema en nodos que se comunican por topicos. Eso permite reutilizar paquetes como AMCL o Nav2, probar en simulacion antes de usar el robot real, y ver lo que pasa con herramientas como RViz.

Limitaciones observadas

- El nodo reactivo solo revisa el frente. Un obstaculo lateral puede provocar un choque, como ocurrio en una de las pruebas.
- No tiene memoria: si el espacio libre es estrecho, puede girar sin salir de la zona.
- La velocidad cambia de forma brusca entre avanzar, frenar y girar.